

Curso de JavaScript Do Básico ao Avançado

Uma jornada completa pela linguagem JavaScript, com teoria clara, prática real e projetos interativos.



Autor: Profº Jefferson Riper

Versão: 2025

Sumário

1. Aula 1 – Introdução ao JavaScript e Sintaxe Básica
2. Aula 2 – Operadores Aritméticos, Lógicos e de Comparação
3. Aula 3 – Controle de Fluxo: Condições (if, else, switch)
4. Aula 4 – Estruturas de Repetição (for, while, do...while)
5. Aula 5 – Funções: Definição, Parâmetros e Retorno
6. Aula 6 – Arrays: Criação, Acesso e Métodos Básicos
7. Aula 7 – Objetos e Manipulação de Propriedades
8. Aula 8 – Manipulação do DOM (Document Object Model)
9. Aula 9 – Eventos no Navegador e Interatividade
10. Aula 10 – Projeto Prático: Calculadora Simples com HTML + JS
11. Aula 11 – Escopo e Hoisting no JavaScript
12. Aula 12 – Closures e Funções Aninhadas
13. Aula 13 – Funções de Alta Ordem (Higher-Order Functions)
14. Aula 14 – Manipulação de Arrays com map, filter e reduce
15. Aula 15 – Introdução ao Assíncronismo com setTimeout e setInterval
16. Aula 16 – Promises e async/await
17. Aula 17 – Manipulação de JSON
18. Aula 18 – Consumindo APIs com fetch
19. Aula 19 – Tratamento de Erros com try, catch e finally
20. Aula 20 – Mini-Projeto: Catálogo de Produtos com API Falsa
21. Aula 21 – Escopo Léxico e Contexto de Execução (this)
22. Aula 22 – Funções Arrow vs Funções Tradicionais
23. Aula 23 – Módulos em JavaScript (ES Modules)
24. Aula 24 – Manipulação Avançada de Arrays
25. Aula 25 – Funções Recursivas
26. Aula 26 – Debounce e Throttle (Controle de Execução)
27. Aula 27 – Prototypes, __proto__ e Herança
28. Aula 28 – Classes em JavaScript (ES6)
29. Aula 29 – Expressões Regulares
30. Aula 30 – Projeto Final – Aplicação com Integração de Módulos e API

Aula 1 – Introdução ao JavaScript e Sintaxe Básica

Objetivos da Aula

- Entender o que é JavaScript e sua importância na Web.
- Aprender a executar código JavaScript no navegador.
- Conhecer os tipos primitivos e como declarar variáveis.
- Escrever os primeiros scripts usando console e HTML.

O que é JavaScript?

JavaScript é uma linguagem de programação usada para tornar páginas da web interativas. É executada diretamente no navegador e permite responder a ações do usuário, modificar elementos na página, validar formulários, criar animações, entre muitas outras possibilidades.

Onde o JavaScript é executado?

JavaScript é executado no navegador. Para começar, você pode abrir o console do navegador (geralmente com a tecla F12) e digitar seus comandos na aba 'Console'.

Exemplo: Usando o Console

Abra o navegador, pressione F12 e vá até a aba 'Console'. Digite o seguinte comando:

```
console.log("Olá, mundo!");
```

Estrutura HTML com JavaScript

Você pode inserir JavaScript diretamente em uma página HTML usando a tag `<script>`. Veja o exemplo abaixo:

```
<!DOCTYPE html>
<html>
<head><title>Meu JS</title></head>
<body>
<h1>Bem-vindo</h1>
<script>
```

```
alert('Olá, mundo!');  
console.log('JS carregado');  
</script>  
</body>  
</html>
```

Variáveis e Tipos Primitivos

Em JavaScript, usamos 'let', 'const' ou 'var' para declarar variáveis. Veja os principais tipos de dados:

- String (texto): let nome = "João";
- Number (número): let idade = 25;
- Boolean (verdadeiro/falso): let ativo = true;
- Null: valor nulo intencionalmente.
- Undefined: valor não definido.

Exemplo Prático

```
let nome = "Ana";  
let idade = 22;  
let estudante = true;
```

```
console.log("Nome:", nome);  
console.log("Idade:", idade);  
console.log("Estuda?", estudante);
```

Exercícios

1. Crie uma variável chamada 'produto' com o nome de um item de mercado.
2. Crie outra variável 'preco' com o valor numérico do item.
3. Use console.log para imprimir a seguinte frase:

"O produto [produto] custa R\$[preco]"
4. Altere o valor de 'preco' e imprima novamente.
5. (Desafio) Crie um script HTML que mostre essa informação em um alerta na tela.

Aula 2 – Operadores Aritméticos, Lógicos e de Comparação

Operadores Aritméticos

Permitem realizar cálculos matemáticos: +, -, *, /, %.

Operadores de Comparação

Comparam valores e retornam verdadeiro ou falso: ==, ===, !=, >, <, >=, <=.

Operadores Lógicos

Permitem combinar condições booleanas: && (E), || (OU), ! (NÃO).

Exemplos Práticos

```
let a = 10;
let b = 5;
console.log(a + b); // Soma
console.log(a > b); // true

let idade = 20;
let maiorDeldade = idade >= 18 && idade < 60;
console.log(maiorDeldade);
```

Exercícios

1. Crie duas variáveis numéricas e exiba a soma entre elas.
2. Verifique se uma variável idade é maior ou igual a 18.
3. Combine duas expressões usando && e exiba o resultado.

Aula 3 – Controle de Fluxo: Condições (if, else, switch)

If e Else

Permitem executar blocos diferentes de código com base em condições.

Switch

Usado para comparar um valor com múltiplos casos.

Exemplos Práticos

```
let nota = 7;
if (nota >= 6) {
  console.log("Aprovado");
}
```

```
} else {  
  console.log("Reprovado");  
}  
  
let cor = 'vermelho';  
switch (cor) {  
  case 'azul': console.log('Cor azul'); break;  
  case 'vermelho': console.log('Cor vermelha'); break;  
  default: console.log('Cor desconhecida');  
}
```

Exercícios

1. Crie um programa que verifique se uma pessoa pode votar (idade ≥ 16).
2. Use switch para exibir uma mensagem diferente para cada dia da semana.

Aula 4 – Estruturas de Repetição (for, while, do...while)

For

Usado quando se sabe o número de repetições. Exemplo: for (let i = 0; i < 5; i++)

While

Executa enquanto a condição for verdadeira.

Do...While

Executa ao menos uma vez antes de verificar a condição.

Exemplos Práticos

```
for (let i = 1; i <= 5; i++) {  
  console.log("Número:", i);  
}
```

```
let i = 0;  
while (i < 3) {  
  console.log(i);  
  i++;  
}
```

```
let j = 0;  
do {
```

```
console.log(j);  
j++;  
} while (j < 2);
```

Exercícios

1. Use um laço for para exibir os números de 1 a 10.
2. Use while para somar números de 1 a 5.
3. Crie um do...while que peça uma senha até que seja '1234'.

Aula 5 – Funções: Definição, Parâmetros e Retorno

Funções

São blocos de código reutilizáveis. Podem receber parâmetros e retornar valores.

Exemplos Práticos

```
function saudacao(nome) {  
  return "Olá, " + nome + "!";  
}  
console.log(saudacao("Maria"));
```

```
const soma = (a, b) => a + b;  
console.log(soma(2, 3));
```

Exercícios

1. Crie uma função que receba dois números e retorne a multiplicação entre eles.
2. Faça uma função que receba um nome e exiba uma saudação personalizada.

Aula 6 – Arrays: Criação, Acesso e Métodos Básicos

Arrays

São listas de valores acessadas por índices.

Métodos

push, pop, shift, unshift, length, indexOf

💡 Exemplos Práticos

```
let frutas = ["maçã", "banana"];  
frutas.push("laranja");  
console.log(frutas);
```

```
let numeros = [10, 20, 30];  
console.log(numeros[1]);
```

📁 Exercícios

1. Crie um array com 3 cores e adicione uma nova cor.
2. Remova o primeiro item do array usando shift.

Aula 7 – Objetos e Manipulação de Propriedades

O que são Objetos?

Objetos em JavaScript são coleções de propriedades, que consistem em pares de chave e valor. Eles permitem representar entidades do mundo real de forma estruturada.

Criando um Objeto

Um objeto pode ser criado com chaves {} e preenchido com propriedades.
Exemplo: `let carro = {marca: 'Fiat', modelo: 'Uno'};`

Acessando e Modificando Propriedades

Podemos acessar com ponto (`obj.prop`) ou colchete (`obj['prop']`). Também é possível adicionar ou alterar valores com essas notações.

💡 Exemplos Práticos

```
let pessoa = {  
  nome: "Carlos",  
  idade: 28,  
  profissao: "Professor"  
};  
console.log(pessoa.nome); // Carlos  
  
pessoa.idade = 29;  
console.log(pessoa);
```

📌 Exercícios

1. Crie um objeto chamado aluno com propriedades nome, idade e curso.
2. Altere o curso do aluno e exiba o objeto atualizado.
3. Adicione uma nova propriedade chamada 'matriculado' com valor booleano.

Aula 8 – Manipulação do DOM (Document Object Model)

O que é o DOM?

O DOM representa a estrutura de uma página HTML como uma árvore de objetos manipuláveis com JavaScript.

Selecionando Elementos

Podemos selecionar elementos com métodos como getElementById, querySelector e querySelectorAll.

Modificando o DOM

É possível alterar o conteúdo, classes, estilos e atributos de elementos HTML usando JavaScript.

💡 Exemplos Práticos

```
document.getElementById("titulo").textContent = "Novo Título";  
  
let paragrafo = document.querySelector("p");  
paragrafo.style.color = "blue";
```

■ Exercícios

1. Altere o texto de um elemento h1 via JavaScript.
2. Mude a cor de fundo de um parágrafo ao carregar a página.
3. Adicione uma classe CSS a um botão existente.

Aula 9 – Eventos no Navegador e Interatividade

O que são Eventos?

Eventos são ações que ocorrem na página, como cliques, digitação ou carregamento. JavaScript pode reagir a esses eventos.

addEventListener

Este método permite escutar eventos e executar funções específicas quando eles ocorrem.

Tipos Comuns de Eventos

click, input, submit, mouseover, keydown, entre outros.

💡 Exemplos Práticos

```
document.querySelector("#botao").addEventListener("click", function() {  
    alert("Botão clicado!");  
});
```

```
document.querySelector("input").addEventListener("input", function(e) {  
    console.log("Valor:", e.target.value);  
});
```

■ Exercícios

1. Faça um botão que, ao ser clicado, mude o texto de um parágrafo.
2. Capture o texto digitado em um input e mostre em tempo real abaixo dele.
3. Ao passar o mouse sobre uma imagem, troque sua borda.

Aula 10 – Projeto Prático: Calculadora Simples com HTML + JS

Objetivo do Projeto

Criar uma calculadora funcional que realiza operações de soma, subtração, multiplicação e divisão.

Estrutura HTML

A interface inclui dois campos de entrada, botões de operação e um campo para o resultado.

Lógica com JavaScript

Cada botão realiza a operação correspondente com os valores dos inputs e mostra o resultado na tela.

Exemplos Práticos

```
<input id='n1'> + <input id='n2'>
<button onclick='somar()'>Somar</button>
<p id='resultado'></p>
<script>
function somar() {
  let a = parseFloat(document.getElementById('n1').value);
  let b = parseFloat(document.getElementById('n2').value);
  document.getElementById('resultado').textContent = a + b;
}
</script>
```

Exercícios

1. Implemente os botões de subtração, multiplicação e divisão.
2. Adicione validação para garantir que os campos não estejam vazios.
3. Estilize a interface com CSS básico.

JavaScript Básico

Apostila Interativa – Aula 1

Aula 11 – Escopo e Hoisting no JavaScript

Objetivos

- Entender as diferenças entre escopos: global, função e bloco.
- Compreender o comportamento de hoisting com var, let, const e funções.
- Identificar problemas comuns e aplicar boas práticas para evitar bugs silenciosos.

Teoria Avançada

Escopo

Escopo define onde uma variável pode ser usada. Existem três tipos principais: global, função e bloco.

- Escopo Global: variáveis declaradas fora de qualquer função.
- Escopo de Função: válidas apenas dentro da função.
- Escopo de Bloco: criadas com let e const dentro de blocos como if, for, while.

Hoisting

Hoisting é o comportamento interno do JavaScript que move declarações para o topo do escopo durante a fase de compilação.

- var: içado com valor undefined.
- let/const: içado mas não inicializado (gera erro).
- function: içada com seu corpo.

Exemplos Reais e Comentados

```
console.log(nome); // undefined
var nome = "Maria";
```

```
console.log(idade); // ReferenceError
let idade = 30;
```

```
saudar(); // Olá!
function saudar() {
  console.log("Olá!");
}
```

```
console.log(x); // undefined
var x = 10;
```

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
} // 3, 3, 3
```

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 100);  
} // 0, 1, 2
```

Exercícios Propostos

1. Explique a diferença entre escopo de função e de bloco.
2. Analise o código a seguir e diga o que será impresso:

```
var x = 1;  
function teste() {  
  console.log(x);  
  var x = 2;  
  console.log(x);  
}  
teste();
```

3. Reescreva o seguinte código com let para evitar problemas de hoisting:

```
function exemplo() {  
  if (true) {  
    var nome = "Ana";  
  }  
  console.log(nome);  
}
```

4. Desafio: explique por que o código abaixo imprime 3, 3, 3 e como corrigir:

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 100);  
}
```

Aula 12 – Closures e Funções Aninhadas

Objetivos

- Compreender o conceito de closures em JavaScript.
- Saber como funções aninhadas compartilham escopos.
- Aplicar closures em situações reais como encapsulamento de dados e criação de funções especializadas.

Teoria Detalhada

Closures são funções que conseguem acessar variáveis externas mesmo depois que o escopo em que foram criadas foi encerrado.

Isso é possível porque a função guarda uma referência ao ambiente léxico onde foi declarada.

♦ Funções Aninhadas

Funções podem ser definidas dentro de outras. A função interna tem acesso total às variáveis da externa.

♦ Closure

Quando retornamos uma função interna, ela mantém o acesso às variáveis da função onde foi criada.

Exemplos Reais e Comentados

Exemplo 1: Função aninhada

```
function externa() {  
  let mensagem = "Olá do escopo externo";  
  function interna() {  
    console.log(mensagem);  
  }  
  interna();  
}
```

Exemplo 2: Closure

```
function contador() {  
  let valor = 0;  
  return function() {  
    valor++;  
    return valor;  
  };  
}  
  
const incrementar = contador();  
console.log(incrementar()); // 1  
console.log(incrementar()); // 2
```

Exercícios Propostos

1. Crie uma função `saudacao` que retorna outra função com um nome como argumento e imprime: "Olá, [nome]!".
2. Construa uma função chamada `multiplicador(x)` que retorna outra função. Esta deve multiplicar o valor recebido por `x`.

3. Explique com suas palavras o que é um closure e por que ele é útil.
4. Qual será o resultado do código abaixo?

```
function criarContador() {  
  let i = 0;  
  return function() {  
    return ++i;  
  }  
}  
  
let c1 = criarContador();  
let c2 = criarContador();  
console.log(c1()); // ?  
console.log(c1()); // ?  
console.log(c2()); // ?
```

Aula 13 – Funções de Alta Ordem (Higher-Order Functions)

🎯 Objetivos

- Entender o que são funções de alta ordem.
- Saber identificar e criar funções que recebem ou retornam outras funções.
- Aplicar o conceito na construção de funções reutilizáveis e abstratas.

🧠 Teoria Detalhada

Funções de alta ordem são funções que:

- Recebem uma ou mais funções como argumentos (funções callback);
- Retornam outra função como resultado.

São muito usadas em JavaScript por serem compatíveis com o paradigma funcional.

Exemplos reais: `setTimeout`, `Array.map`, `Array.filter`, `Array.reduce`.

💡 Função que recebe outra função

Permite abstrair ações e definir comportamentos reutilizáveis. A função callback é executada dentro da função de alta ordem.

💡 Função que retorna outra função

Isso permite criar funções especializadas, mantendo dados privados e reduzindo repetição de código (muito usado com closures).

💡 Exemplos Reais e Comentados

Exemplo 1: Função que recebe função

```
function executar(fn) {  
  console.log("Início");  
  fn();  
  console.log("Fim");  
}  
executar(() => console.log("Função executada"));
```

Exemplo 2: Função que retorna outra

```
function multiplicador(fator) {  
  return function(numero) {  
    return numero * fator;  
  }  
}  
const dobrar = multiplicador(2);  
console.log(dobrar(5)); // 10
```

Exemplo 3: Abstração com callback

```
function aplicarOperacao(valor, operacao) {  
  return operacao(valor);  
}  
const triplo = n => n * 3;  
console.log(aplicarOperacao(4, triplo)); // 12
```

📌 Exercícios Propostos

1. Crie uma função chamada `executarDuasVezes` que recebe uma função como argumento e a executa duas vezes.
2. Construa uma função chamada `criarSaudacao(saudacao)` que retorna outra função que recebe o nome da pessoa.
3. Escreva uma função `calcular` que recebe dois números e uma função de operação (como soma, subtração etc.).
4. (Desafio) Implemente uma função `compose(f, g)` que retorna uma nova função onde `f(g(x))` é executado.

Aula 14 – Manipulação de Arrays com map, filter e reduce

Objetivos

- Entender o propósito de `map`, `filter` e `reduce`.
- Aprender como transformar, filtrar e agregar dados em arrays.
- Utilizar esses métodos em cenários do dia a dia.

Teoria Detalhada

`map`, `filter` e `reduce` são métodos da linguagem JavaScript aplicados sobre arrays. Eles são utilizados para transformar, selecionar ou agregar dados de forma funcional e elegante.

♦ **map()**

Cria um novo array com os resultados da aplicação de uma função a cada item do array original.

Sintaxe: `novoArray = array.map(callback)`

♦ **filter()**

Retorna um novo array apenas com os elementos que satisfazem uma condição booleana.

Sintaxe: `novoArray = array.filter(callback)`

♦ **reduce()**

Reduz o array a um único valor acumulando os elementos.

Sintaxe: `valorFinal = array.reduce(callback, valorInicial)`

Exemplos Reais e Comentados

Exemplo 1 – map(): dobrando os valores

```
const numeros = [1, 2, 3];  
const dobrados = numeros.map(n => n * 2);  
console.log(dobrados); // [2, 4, 6]
```

Exemplo 2 – filter(): apenas números pares

```
const numeros = [1, 2, 3, 4];  
const pares = numeros.filter(n => n % 2 === 0);  
console.log(pares); // [2, 4]
```

Exemplo 3 – reduce(): somando valores

```
const numeros = [1, 2, 3, 4];  
const soma = numeros.reduce((acumulador, atual) => acumulador + atual, 0);  
console.log(soma); // 10
```

```
# Exemplo 4 – Uso combinado (map + filter)
const nomes = ["Ana", "Carlos", "Beatriz"];
const nomesComA = nomes.filter(nome => nome.includes("a")).map(n =>
n.toUpperCase());
console.log(nomesComA); // ["ANA", "BEATRIZ"]
```

Exercícios Propostos

1. Dado um array de números, use `map` para criar um novo array com o quadrado de cada número.

2. Use `filter` para extrair os nomes com mais de 5 letras do array:

```
const nomes = ['João', 'Fernanda', 'Carlos', 'Eva'];
```

3. Some todos os elementos de um array de preços com `reduce`:

```
const precos = [10.5, 22.3, 18.4];
```

4. (Desafio) Dado um array de produtos com nome e preço, use `reduce` para calcular o valor total da compra:

```
const carrinho = [
  { nome: 'Camiseta', preco: 29.99 },
  { nome: 'Calça', preco: 89.9 },
  { nome: 'Meias', preco: 9.99 }
];
```

Aula 15 – Introdução ao Assíncronismo com `setTimeout` e `setInterval`

Objetivos

- Compreender o conceito de assíncronismo em JavaScript.
- Aprender como utilizar `setTimeout` e `setInterval` para controlar a execução no tempo.
- Identificar quando usar cada função e como parar execuções contínuas.

Teoria Detalhada

JavaScript é uma linguagem de execução **monothread** e assíncrona, baseada em **event loop**. Isso significa que, embora tenha apenas uma thread de execução, é capaz de reagir a eventos e realizar tarefas de forma

não bloqueante.

Duas funções fundamentais para isso são:

- `setTimeout(fn, tempo)`: executa a função `fn` uma única vez após o tempo (em milissegundos).
- `setInterval(fn, tempo)`: executa a função `fn` repetidamente a cada intervalo definido.

💡 Exemplos Reais e Comentados

Exemplo 1 – Mensagem com atraso

```
setTimeout(() => {  
  console.log("Executado após 2 segundos");  
}, 2000);
```

Exemplo 2 – Repetição com `setInterval`

```
let contador = 0;  
const intervalo = setInterval(() => {  
  console.log("Contador:", ++contador);  
  if (contador === 5) clearInterval(intervalo);  
}, 1000);
```

Exemplo 3 – Parar timeout com `clearTimeout`

```
const id = setTimeout(() => console.log("Não será executado"), 3000);  
clearTimeout(id);
```

📌 Exercícios Propostos

1. Use `setTimeout` para exibir "Olá, mundo!" após 1,5 segundos.
2. Crie um contador que exibe um número a cada segundo até 10 e depois para.
3. Faça uma função que, ao ser chamada, inicie um `setInterval` e outra que pare esse intervalo.
4. (Desafio) Implemente um cronômetro com `setInterval`, mostrando minutos e segundos.

Aula 16 – Promises e async/await

Objetivos

- Compreender o que são Promises e sua função no controle de fluxo assíncrono.
- Aprender a lidar com tarefas que levam tempo, como requisições HTTP.
- Usar `async` e `await` para escrever código assíncrono mais limpo e legível.

Teoria Detalhada

Em JavaScript moderno, o assíncronismo é tratado com Promises. Uma Promise representa um valor que pode estar disponível agora, no futuro ou nunca.

- Estados de uma Promise: **pending**, **fulfilled**, **rejected**.
- Métodos principais: `then()`, `catch()`, `finally()`.

Com `async/await`, podemos escrever código assíncrono que parece síncrono, facilitando a leitura e o tratamento de erros.

Exemplos Reais e Comentados

Exemplo 1 – Promise básica

```
const promessa = new Promise((resolve, reject) => {  
  setTimeout(() => resolve("Concluído"), 2000);  
});  
promessa.then(res => console.log(res));
```

Exemplo 2 – Tratando erro

```
new Promise((resolve, reject) => {  
  setTimeout(() => reject("Erro!"), 1000);  
}).catch(err => console.error(err));
```

Exemplo 3 – Async/Await

```
function esperar(ms) {  
  return new Promise(resolve => setTimeout(resolve, ms));  
}
```

```
async function executar() {  
  console.log("Início");  
  await esperar(2000);  
  console.log("Fim após 2 segundos");  
}  
executar();
```

Exercícios Propostos

1. Crie uma Promise que resolva com uma mensagem após 3 segundos.

2. Use `.then()` para mostrar a mensagem no console.
3. Reescreva o exercício anterior usando `async` e `await`.
4. (Desafio) Crie uma função `carregarDados()` que simula uma requisição assíncrona com delay e trate erro com `try/catch`.

Aula 17 – Manipulação de JSON

Objetivos

- Compreender o formato JSON e sua importância na troca de dados entre sistemas.
- Aprender a converter objetos JavaScript em JSON e vice-versa.
- Manipular e acessar dados JSON de forma segura e eficiente.

Teoria Detalhada

JSON (JavaScript Object Notation) é um formato leve de troca de dados, amplamente utilizado em APIs.

- É baseado em texto, fácil de ler e escrever.
- Muito usado para enviar/receber informações entre cliente e servidor.

Em JavaScript, manipulamos JSON com:

- `JSON.stringify(obj)` → Converte objeto JavaScript em string JSON.
- `JSON.parse(json)` → Converte string JSON em objeto JavaScript.

Exemplos Reais e Comentados

Exemplo 1 – Converter objeto para JSON

```
const pessoa = { nome: "João", idade: 30 };
const json = JSON.stringify(pessoa);
console.log(json); // '{"nome":"João","idade":30}'
```

Exemplo 2 – Converter JSON para objeto

```
const texto = '{"nome":"Maria","idade":25}';
const obj = JSON.parse(texto);
console.log(obj.nome); // Maria
```

Exemplo 3 – Usando JSON para enviar dados (simulação)

```
const produto = { id: 1, nome: "Camisa" };
fetch("https://api.exemplo.com/produtos", {
  method: "POST",
```

```
headers: { "Content-Type": "application/json" },  
body: JSON.stringify(produto)  
});
```

Exercícios Propostos

1. Crie um objeto representando um aluno com nome, idade e curso.
2. Converta esse objeto para JSON e exiba no console.
3. Agora transforme uma string JSON válida em objeto e acesse uma das propriedades.
4. (Desafio) Simule o envio de um objeto representando um pedido (produto, quantidade, valor) usando `fetch` e `JSON.stringify`.

Aula 18 – Consumindo APIs com fetch

Objetivos

- Compreender como o método `fetch` é utilizado para fazer requisições HTTP.
- Aprender a manipular respostas de APIs REST utilizando JSON.
- Praticar requisições GET e POST com tratamento de respostas e erros.

Teoria Detalhada

A função `fetch` é nativa do JavaScript moderno e permite fazer chamadas HTTP para servidores ou APIs.

Ela retorna uma Promise que resolve para um objeto `Response`, que pode ser convertido em JSON, texto, blob etc.

****Sintaxe básica:****

```
``javascript  
fetch(url, opções)  
  .then(resposta => resposta.json())  
  .then(dados => console.log(dados))  
  .catch(erro => console.error(erro));  
``
```

Exemplos Reais e Comentados

Exemplo 1 – Requisição GET simples

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
```

```

.then(res => res.json())
.then(dados => console.log(dados))
.catch(err => console.error("Erro:", err));

# Exemplo 2 – Requisição POST com JSON
const novoPost = {
  title: "Meu Post",
  body: "Conteúdo do post",
  userId: 1
};

fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(novoPost)
})
.then(res => res.json())
.then(dados => console.log("Criado:", dados));

```

Exercícios Propostos

1. Use `fetch` para buscar os dados de um usuário do JSONPlaceholder (`/users/1`) e imprima o nome.
2. Faça uma requisição POST simulando o envio de um comentário.
3. (Desafio) Monte uma função que receba um `endpoint` e retorne os dados obtidos via fetch.
4. (Extra) Combine `async/await` com `fetch` para buscar dados de uma API pública.

Aula 19 – Tratamento de Erros com try, catch e finally

Objetivos

- Compreender como lidar com exceções e erros em JavaScript.
- Aprender a usar blocos `try`, `catch` e `finally` para controle de falhas.
- Implementar códigos mais robustos e seguros.

Teoria Detalhada

Em JavaScript, erros podem ocorrer durante a execução do código. O tratamento adequado desses erros é essencial para evitar falhas não controladas e melhorar a experiência do usuário.

A estrutura `try...catch` permite capturar exceções que ocorrem em blocos de código potencialmente problemáticos. O `finally` sempre será executado, mesmo que ocorra um erro ou não.

****Sintaxe:****

```
```\javascript
try {
 // código que pode gerar erro
} catch (erro) {
 // tratamento do erro
} finally {
 // sempre executa
}
...`
```

## Exemplos Reais e Comentados

# Exemplo 1 – Capturando erro

```
try {
 const resultado = JSON.parse("{ inválido }");
} catch (erro) {
 console.error("Erro ao analisar JSON:", erro.message);
}
```

# Exemplo 2 – Uso de finally

```
try {
 console.log("Executando...");
} catch (e) {
 console.error("Erro detectado");
} finally {
 console.log("Finalizado com ou sem erro");
}
```

# Exemplo 3 – Função robusta com tratamento

```
function dividir(a, b) {
 try {
 if (b === 0) throw new Error("Divisão por zero");
 return a / b;
 }
```



```
} catch (e) {
 return e.message;
}
}
console.log(dividir(10, 0)); // "Divisão por zero"
```

### Exercícios Propostos

1. Crie um código que tente converter uma string JSON inválida e trate o erro com ``catch``.
2. Implemente uma função ``dividirSegura(a, b)`` que trate divisão por zero.
3. Use ``finally`` para mostrar uma mensagem de encerramento, independentemente de erro.
4. (Desafio) Combine ``fetch`` com ``try/catch`` e trate erros de rede ou falhas na conversão de JSON.

## Aula 20 – Mini-Projeto: Catálogo de Produtos com API Falsa

### Objetivos

- Aplicar conhecimentos de DOM, fetch, JSON e tratamento de erros.
- Construir uma interface simples que consome uma API e exibe dados dinamicamente.
- Estimular a prática com projeto completo e funcional.

### Descrição do Projeto

Neste mini-projeto, vamos criar um catálogo de produtos consumindo uma API falsa (FakeStoreAPI). O objetivo é buscar os produtos da API e exibi-los em cards com nome, imagem, descrição e preço.

### Ferramentas Utilizadas

- HTML básico para estrutura da página
- JavaScript com ``fetch`` para consumo de API
- Manipulação de DOM para exibir os produtos
- Tratamento de erros com ``try``, ``catch``

## 💡 Código Completo do Projeto

```
<!DOCTYPE html>
<html>
<head>
 <title>Catálogo de Produtos</title>
 <style>
 .produto { border: 1px solid #ccc; padding: 10px; margin: 10px; width: 200px; }
 img { max-width: 100%; }
 </style>
</head>
<body>

<h1>Catálogo</h1>
<div id="produtos"></div>

<script>
async function carregarProdutos() {
 try {
 const resposta = await fetch("https://fakestoreapi.com/products");
 const dados = await resposta.json();
 const container = document.getElementById("produtos");
 dados.forEach(produto => {
 const div = document.createElement("div");
 div.className = "produto";
 div.innerHTML = `
 <h2>${produto.title}</h2>

 <p>${produto.description}</p>
 R$ ${produto.price}
 `;
 container.appendChild(div);
 });
 } catch (erro) {
 console.error("Erro ao carregar produtos:", erro);
 }
}
carregarProdutos();
</script>

</body>
</html>
```

## Desafios Propostos

1. Adicione um botão "Carregar mais" que busca os próximos 5 produtos (simular).
2. Estilize o catálogo com CSS para exibir os produtos em grade (grid).
3. Adicione uma barra de busca para filtrar os produtos por nome.
4. (Avançado) Permita ordenar os produtos por preço usando um seletor.

JavaScript Básico

Apostila Interativa – Aula 1

## Aula 21 – Escopo Léxico e Contexto de Execução (`this`)

### Objetivos

- Compreender o que é escopo léxico e como o JavaScript resolve identificadores.
- Entender o funcionamento do `this` em diferentes contextos: global, função, método, arrow function.
- Evitar armadilhas comuns com `this` e aplicar boas práticas.

### Teoria Detalhada

#### Escopo Léxico

Escopo léxico significa que a visibilidade das variáveis é determinada pela estrutura do código no momento da definição. Funções têm acesso às variáveis do escopo onde foram declaradas, mesmo que sejam executadas fora dele.

#### Contexto de Execução e `this`

O valor de `this` depende de **como** a função é chamada. Pode variar em diferentes situações:

- No escopo global: `this` referencia o objeto global (`window` no navegador).
- Em métodos de objetos: `this` referencia o objeto ao qual o método pertence.
- Em funções comuns chamadas isoladamente: `this` é o objeto global (modo não estrito).
- Em `arrow functions`: `this` é **lexicamente herdado**, ou seja, não se refere ao contexto de chamada, mas ao de definição.

## 💡 Exemplos Reais e Comentados

```
Escopo léxico
function externa() {
 const mensagem = "Olá";
 function interna() {
 console.log(mensagem); // acessa a variável do escopo pai
 }
 interna();
}
```

```
this em objeto
const pessoa = {
 nome: "Ana",
 saudacao: function() {
 console.log("Olá, meu nome é " + this.nome);
 }
};
pessoa.saudacao(); // Ana
```

```
this em função comum (fora de objeto)
function mostrar() {
 console.log(this);
}
mostrar(); // window (no navegador)
```

```
Arrow function herda o this
const obj = {
 nome: "Lucas",
 falar: () => {
 console.log(this.nome); // undefined
 }
};
obj.falar();
```

## 📌 Exercícios Propostos

1. Explique com suas palavras o que é escopo léxico.
2. Dado um objeto com um método que usa `this`, observe o valor retornado:

```
const user = {
 nome: "João",
 mostrar: function() {
 console.log(this.nome);
 }
};
```

```
}
};
user.mostrar();
```

3. Reescreva o método acima com arrow function. O que acontece com `this`?
4. (Desafio) Crie uma função que retorna uma função interna e demonstra escopo léxico usando `this` corretamente.

## Aula 22 – Funções Arrow vs Funções Tradicionais

### Objetivos

- Entender as diferenças de sintaxe e comportamento entre arrow functions e funções tradicionais.
- Identificar quando usar cada uma.

### Teoria Detalhada

Arrow functions possuem sintaxe reduzida e não possuem seu próprio `this`. São ideais para callbacks e funções curtas.

### Exemplos Reais e Comentados

```
const soma = (a, b) => a + b;
```

```
function somaTradicional(a, b) {
 return a + b;
}
```

### Exercícios Propostos

1. Converta funções tradicionais em arrow functions.
2. Explique quando evitar arrow functions com `this`.

## Aula 23 – Módulos em JavaScript (ES Modules)

### Objetivos

- Aprender a exportar/importar funções e objetos usando `export` e `import`.
- Modularizar código para melhor organização.

### Teoria Detalhada

ES Modules permitem dividir código em arquivos reutilizáveis. Use `export` para tornar funções acessíveis e `import` para utilizá-las.

### Exemplos Reais e Comentados

```
// arquivo utils.js
export function saudacao(nome) { return `Olá, ${nome}`; }

// arquivo app.js
import { saudacao } from './utils.js';
```

### Exercícios Propostos

1. Crie dois arquivos e compartilhe funções entre eles usando ES Modules.
2. Faça um módulo que exporta constantes e outro que importa e usa.

## Aula 24 – Manipulação Avançada de Arrays

### Objetivos

- Explorar métodos como `some`, `every`, `find`, `sort`, `flatMap`.
- Aplicar lógica condicional e transformação em arrays.

### Teoria Detalhada

Além de `map`, `filter`, `reduce`, JavaScript oferece métodos úteis para validação, busca e transformação.

### Exemplos Reais e Comentados

```
const numeros = [1, 2, 3, 4];
numeros.some(n => n > 2); // true

const produtos = [
 { nome: 'A', preco: 10 },
 { nome: 'B', preco: 5 }
```

```
];
produtos.sort((a, b) => a.preco - b.preco);
```

### Exercícios Propostos

1. Use `every` para verificar se todos os itens são maiores que 0.
2. Encontre o produto mais barato usando `reduce` ou `sort`.

## Aula 25 – Funções Recursivas

### Objetivos

- Entender como uma função pode chamar a si mesma.
- Resolver problemas de forma recursiva como fatorial, soma e contagem.

### Teoria Detalhada

A recursão é uma técnica onde uma função resolve um problema chamando a si mesma com um subtarefa. É útil quando o problema pode ser dividido em partes menores.

### Exemplos Reais e Comentados

```
function fatorial(n) {
 if (n <= 1) return 1;
 return n * fatorial(n - 1);
}
```

```
function contagem(n) {
 if (n === 0) return;
 console.log(n);
 contagem(n - 1);
}
```

### Exercícios Propostos

1. Implemente uma função recursiva que soma números até 0.
2. Reescreva o cálculo de potência (ex:  $2^3$ ) usando recursão.

## Aula 26 – Debounce e Throttle (Controle de Execução)

### Objetivos

- Aprender técnicas para limitar chamadas de funções com base em tempo.
- Aplicar debounce e throttle em inputs e eventos contínuos.

### Teoria Detalhada

Debounce evita execuções múltiplas até o usuário parar de interagir. Throttle limita a frequência de chamadas.

### Exemplos Reais e Comentados

```
// Debounce
function debounce(fn, delay) {
 let timer;
 return function(...args) {
 clearTimeout(timer);
 timer = setTimeout(() => fn(...args), delay);
 };
}
```

```
// Throttle
function throttle(fn, interval) {
 let last = 0;
 return function(...args) {
 const now = Date.now();
 if (now - last >= interval) {
 last = now;
 fn(...args);
 }
 };
}
```

### Exercícios Propostos

1. Use debounce para evitar múltiplas buscas em tempo real ao digitar.
2. Use throttle para limitar scroll ou resize a 1x por segundo.



## Aula 27 – Prototypes, \_\_proto\_\_ e Herança

### Objetivos

- Compreender o sistema de protótipos do JavaScript.
- Criar objetos com herança de comportamento.

### Teoria Detalhada

Todo objeto em JS herda propriedades e métodos de seu protótipo. `\_\_proto\_\_` aponta para esse objeto protótipo, que pode ser compartilhado.

### Exemplos Reais e Comentados

```
function Pessoa(nome) {
 this.nome = nome;
}
Pessoa.prototype.falar = function() {
 console.log("Oi, " + this.nome);
};
const p1 = new Pessoa("João");
p1.falar();
```

### Exercícios Propostos

1. Crie dois objetos usando função construtora e compartilhe métodos com prototype.
2. Use `Object.create` para herdar de outro objeto base.

## Aula 28 – Classes em JavaScript (ES6)

### Objetivos

- Aprender a sintaxe de classes e como instanciar objetos.
- Compreender herança com `extends` e o uso do `super()`.

### Teoria Detalhada

Classes são uma forma mais clara de escrever construtores e protótipos. Suportam herança direta e encapsulamento.

### Exemplos Reais e Comentados

```
class Animal {
 constructor(nome) {
```

```
 this.nome = nome;
 }
 falar() {
 console.log(this.nome + ' faz barulho');
 }
}
```

```
class Cachorro extends Animal {
 falar() {
 console.log(this.nome + ' late');
 }
}
const rex = new Cachorro('Rex');
rex.falar();
```

### Exercícios Propostos

1. Crie uma classe `Produto` com nome e preço e um método `exibir()`.
2. Crie uma subclasse `Livro` que herda de `Produto` e adiciona o atributo `autor`.

## Aula 29 – Expressões Regulares

### Objetivos

- Aprender a usar RegEx para busca e validação de strings.
- Aplicar padrões com `test()`, `match()`, `replace()`.

### Teoria Detalhada

Regex são padrões usados para testar e extrair informações de strings. Útil para validações, buscas e substituições.

### Exemplos Reais e Comentados

```
const padrao = /[0-9]+/;
console.log(padrao.test("123")); // true
```

```
"email@example.com".match(/[\w.-]+@[\w.-]+\.\w+/);
```

### Exercícios Propostos

1. Valide se uma string contém apenas letras.

2. Encontre todos os números em um texto.
3. (Desafio) Valide um e-mail simples com RegEx.

## Aula 30 – Projeto Final – Aplicação com Integração de Módulos e API

### Objetivos

- Consolidar os aprendizados aplicando ES Modules, fetch, tratamento de erros e DOM.
- Criar uma aplicação modular interativa.

### Teoria Detalhada

Vamos construir uma aplicação de busca de produtos com filtros, consumo de API e estrutura em módulos.

### Exemplos Reais e Comentados

```
// api.js
export async function buscarProdutos() {
 const res = await fetch('https://fakestoreapi.com/products');
 return await res.json();
}
```

### Exercícios Propostos

1. Crie um `index.html` com botão de carregar produtos.
2. Use `api.js` e `ui.js` para separar lógica e visual.
3. Faça a busca de produtos e exiba com imagem e nome.
4. Adicione filtros por categoria ou palavra-chave.